

TEORÍA DE ALGORITMOS
(75.29) CURSO BUCHWALD - GENENDER

Trabajo Práctico: Juegos de Hermanos

10 de enero de 2025

Integrantes:

1. Alejandro Arenas
98199
2. Andrés Moyano
110017
3. Camila Suárez
110800
4. Tomas García
110478
5. Isaías Cabrera
108885

1. Introducción y primeros años

1.1. Análisis del problema

La consigna plantea el siguiente problema: Dados los n valores de todas las monedas, indicar qué monedas debe ir eligiendo Sophia para si misma y para Mateo, siendo que sólo puede elegirla primera de la fila, o bien la última. Se debe asegurar que Sophia gane siempre (considerar que ella siempre comienza primero).

Se plantea un algoritmo Greedy que considera que Mateo no sabe jugar y sofia elije por el. Para asegurar su victoria, Sophia debe elegir la moneda de mayor valor en su turno, y la de menor valor en el turno de Mateo.

El objetivo de Sophia es finalizar con puntaje acumulado mas alto, sabiendo que ella toma las decisiones de ambos. La regla simple elegida para nuestro algoritmo Greedy consiste en elegir en el turno de Sophia la moneda mas grande, que le agregue la mayor rentabilidad actual a la suma de su puntaje, y hacer lo opuesto en el turno de Mateo.

1.1.1. Seguimiento

Se usa el arreglo de monedas [8, 15, 3, 7, 10, 5, 12] como ejemplo.

Empieza jugando Sophia. Como puede elegir entra la primera y la última, sus opciones son 8 y 12: elige 12 ya que es la de mayor valor. El turno siguiente es de Mateo. Como la moneda 12 ya fue elegida, sus opciones ahora son 8 y 5: como Sophia juega por el, elige la menor. Este procedimiento se repite hasta llegar a la última moneda.

- 1° turno, Sophia: entre 8 y 12 elige 12. Queda [8, 15, 3, 7, 10, 5]

Puntaje acumulado Sophia: 12

- 2° turno, Mateo: entre 8 y 5 elige 5. Queda [8, 15, 3, 7, 10]

Puntaje acumulado Mateo: 5

- 3° turno, Sophia: entre 8 y 10 elige 10. Queda [8, 15, 3, 7]

Puntaje acumulado Sophia: 22

- 4° turno, Mateo: entre 8 y 7 elige 7. Queda [8, 15, 3]

Puntaje acumulado Mateo: 12

- 5° turno, Sophia: entre 8 y 3 elige 8. Queda [15, 3]

Puntaje acumulado Sophia: 30

- 6° turno, Mateo: entre 15 y 3 elige 3. Queda [15]

Puntaje acumulado Mateo: 15

- 7° turno, Sophia: elige 15. Queda []

Puntaje acumulado Sophia: 45

- Juego finalizado: 45 puntos para Sophia, 15 para Mateo.

1.2. Demostración del algoritmo

Para demostrar que nuestro algoritmo Greedy es óptimo, debemos demostrar que se encuentra el óptimo local en cada turno.

Lo demostramos por inducción, con un arreglo de tamaño n .

- Caso base: $n = 1$

Si hay solo una moneda en la fila, la elige Sophia y es obviamente óptimo.

- $n > 1$

Suponiendo que el algoritmo funciona de manera óptima para cualquier fila de k monedas, para $k < n$ (k siendo el tamaño del arreglo después de cada turno), consideramos una fila de n monedas: $C_1, C_2, \dots, C_n$. Sophia tiene dos opciones:

1. Tomar la moneda C_1 y dejar C_2 o C_n para el siguiente turno.
2. Tomar la moneda C_n y dejar C_1 o C_{n-1} para el siguiente turno.

En ambos casos Sophia puede predecir el resultado óptimo, ya que tanto ella como Mateo jugarán con estrategias diseñadas por ella misma, para maximizar y minimizar los resultados actuales. En cada turno la moneda agarrada por Sophia será mayor a la moneda de Mateo, por lo que la sumatoria de las monedas acumuladas de Sophia siempre va a ser mayor a la sumatoria de las de Mateo.

Aplicando de manera iterativa nuestra regla, se van obteniendo los óptimos locales, lo que permite eventualmente llegar al óptimo global.

1.3. Algoritmo Greedy

```
1 def iter_greedy(monedas):
2     total_sophia = 0
3     total_mateo = 0
4     turno_sofia = True
5     inicio_actual = 0
6
7     while inicio_actual < len(monedas):
8         if turno_sofia:
9             if monedas[inicio_actual] >= monedas[-1]:
10                 total_sophia += monedas[inicio_actual]
11                 inicio_actual += 1
12             else:
13                 total_sophia += monedas.pop()
14         else:
15             if monedas[inicio_actual] <= monedas[-1]:
16                 total_mateo += monedas[inicio_actual]
17                 inicio_actual += 1
18             else:
19                 total_mateo += monedas.pop()
20
21         turno_sofia = not turno_sofia
22
23     return total_sophia, total_mateo
```

1.3.1. Complejidad

La complejidad del algoritmo propuesto para que Sophia gane el juego siempre es $\mathcal{O}(n)$. En cada iteración, se toma una decisión sobre si tomar la primera o la última moneda. Ambas operaciones

son constantes: Tomar la última moneda: `monedas.pop()` tiene complejidad $\mathcal{O}(1)$. Tomar la primera moneda: Incrementar inicio actual es $\mathcal{O}(n)$. El bucle `while` itera exactamente n veces y en cada iteración elimina una moneda del conjunto de monedas para la siguiente iteración. Esto asegura que el número total de iteraciones es lineal con respecto al número de monedas, n . Estamos recorriendo todos los elementos (n) teniendo complejidad $\mathcal{O}(1)$ cada subproblema en las iteraciones, resultando una complejidad general del algoritmo $\mathcal{O}(n)$.

1.4. Variabilidad de los valores

La variabilidad en los valores de las monedas no afecta los tiempos de ejecución del algoritmo ni a la Optimalidad del Algoritmo, ya que la complejidad depende exclusivamente de la longitud de la lista de monedas, no de los valores de las monedas. El enfoque dinámico garantiza que Sophia maximiza su ganancia independientemente de la magnitud de los valores.

1.5. Ejemplos de ejecución

Además de los ejemplos proporcionados por la cátedra, nosotros diseñamos tres más para probar el algoritmo planteado.

1.5.1. Ej. 1: [50, 200, 150, 300, 100, 250, 400]

Primer turno Sophia: debe elegir entre 50 y 400. Como siempre elige la de mayor valor, se queda con 400.

Primer turno Mateo: debe elegir entre 50 y 250. Como Sophia siempre elige la menor para su hermano, se queda con 50.

Se va repitiendo el procedimiento: 250 para Sophia, 100 para Mateo, 300 para Sophia, 150 para Mateo, 200 para Sophia.

Puntaje total Sophia: $400 + 250 + 300 + 200 = 1150$

Puntaje total Mateo: $50 + 100 + 150 = 300$

1.5.2. Ej. 2: [520, 781, 334, 568, 706, 362, 201, 482, 19, 145]

Primer turno Sophia: debe elegir entre 520 y 145. Como siempre elige la de mayor valor, se queda con 520.

Primer turno Mateo: debe elegir entre 781 y 145. Como Sophia siempre elige la menor para su hermano, se queda con 145.

Se va repitiendo el procedimiento: 781 para Sophia, 19 para Mateo, 482 para Sophia, 201 para Mateo, 362 para Sophia, 334 para Mateo, 706 para Sophia, 568 para Mateo.

Puntaje total Sophia: $520 + 781 + 482 + 362 + 706 = 2851$

Puntaje total Mateo: $145 + 19 + 201 + 334 + 568 = 1267$

1.5.3. Ej. 3: [45, 78, 120, 350, 95, 180, 250, 300, 400, 210, 50, 275, 330, 125, 80, 500]

Primer turno Sophia: debe elegir entre 45 y 500. Como siempre elige la de mayor valor, se queda con 500.

Primer turno Mateo: debe elegir entre 45 y 80. Como Sophia siempre elige la menor para su hermano, se queda con 45.

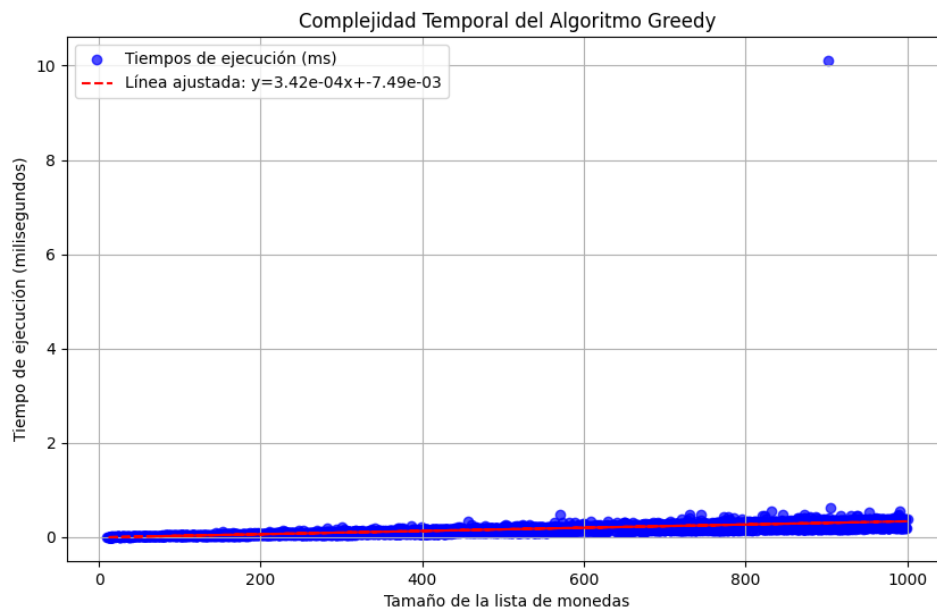
Se va repitiendo el procedimiento: 80 para Sophia, 78 para Mateo, 125 para Sophia, 120 para Mateo, 350 para Sophia, 95 para Mateo, 330 para Sophia, 180 para Mateo, 275 para Sophia, 50 para Mateo, 250 para Sophia, 210 para Mateo, 400 para Sophia, 300 para Mateo.

Puntaje total Sophia: $= 500 + 80 + 125 + 350 + 330 + 275 + 250 + 400 = 2310$

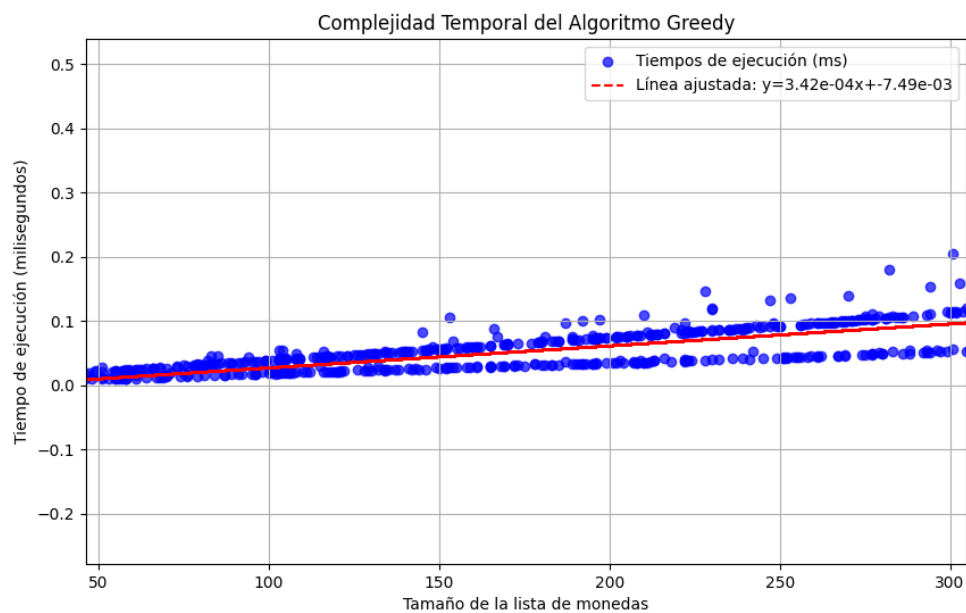
Puntaje total Mateo: $= 45 + 78 + 120 + 95 + 180 + 50 + 210 + 300 = 1078$

1.5.4. Graficos Algoritmo Greedy

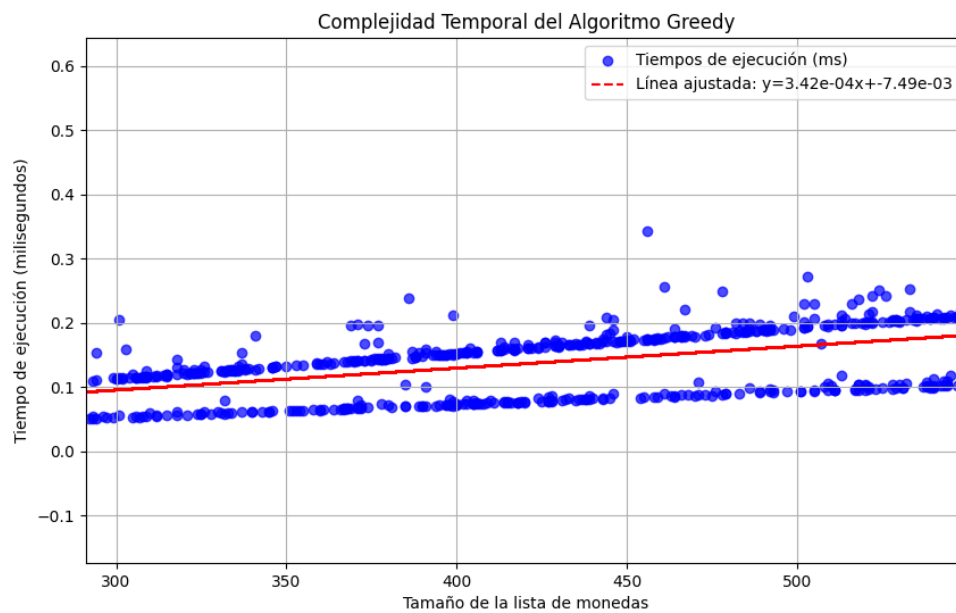
Se crearon 2000 listas de monedas de longitud aleatoria (de 10 a 1000), con contenido también aleatoriamente generado con valor de 1 a 2000. Se observa que el gráfico del algoritmo cumple con ser una función lineal.



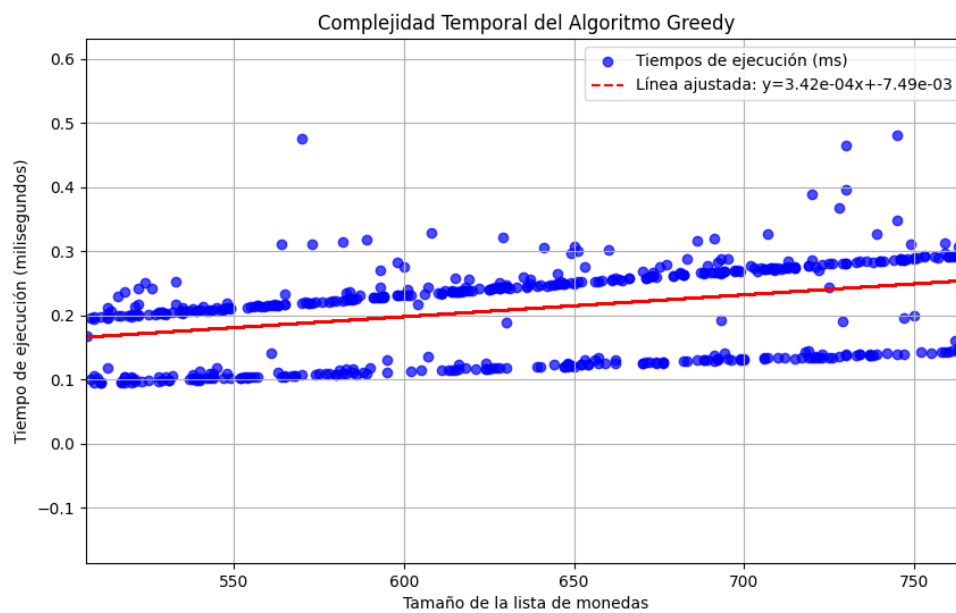
Zoom 1: Listas de 50 a 300 monedas



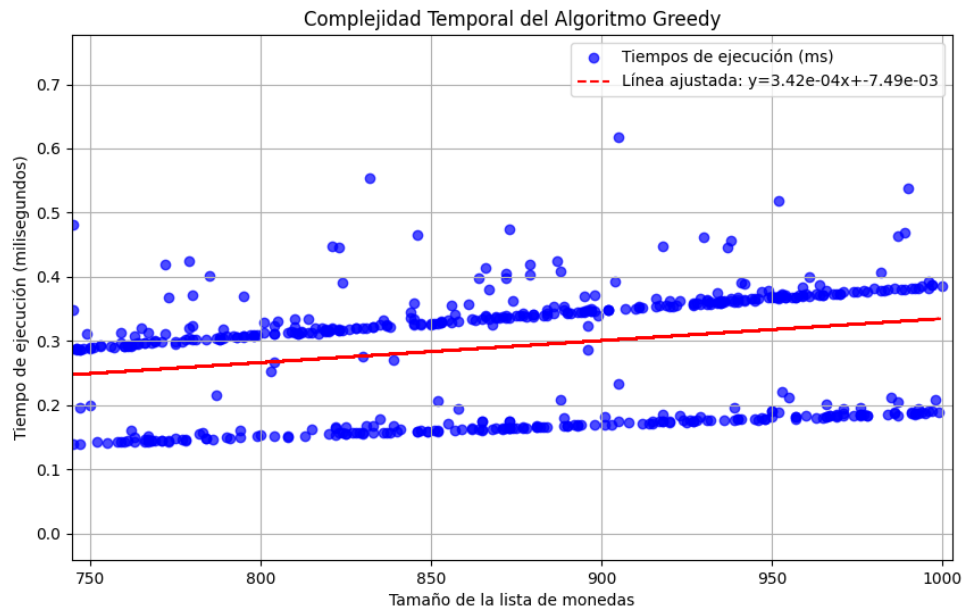
Zoom 2: Listas de 300 a 550 monedas



Zoom 3: Listas de 550 a 750 monedas



Zoom 4: Listas de 750 a 1000 monedas



2. Mateo empieza a Jugar

2.1. Análisis del problema

Mateo ya aprendió a jugar aplicando algoritmos greedy, Sophia también mejoró y ahora aplica programación dinámica.

Planteamos un algoritmo que indique qué moneda debe elegir Sophia para maximizar su puntaje final. Ahora, la moneda de Mateo no es elegida por Sophia, sino que la elige él basándose en un algoritmo Greedy.

Así, Mateo elegirá siempre la moneda de mayor valor, mientras que Sophia va a buscar predecir la mejor opción teniendo en cuenta también las monedas adyacentes a las de los extremos.

Modelamos el problema como una búsqueda del máximo puntaje que Sophia puede obtener, teniendo en cuenta las monedas del arreglo de tamaño n , desde la del índice i hasta el índice j , representado como $dp[i][j]$. La matriz almacena el máximo valor que puede obtener el jugador que empieza, o sea, Sophia.

Nuestro algoritmo usa la ecuación de recurrencia planteada en el punto siguiente y considera las dos opciones posibles en cada turno. Luego, se construye la solución usando bottom-up: La matriz dp se llena empezando con los subproblemas más chicos (intervalos de $n = 1$) y extendiéndose a problemas más grandes (intervalos de longitud n).

- Para intervalos de longitud 1 ($i = j$), $dp[i][j] = C[i]$, ya que hay solo una moneda para elegir.
- Para intervalos de longitud mayor a 1, se calcula $dp[i][j]$ usando la ecuación de recurrencia, iterando sobre los posibles rangos i y j .

Finalmente se reconstruye la solución siguiendo el camino registrado en la matriz dp para determinar qué decisiones fueron tomadas en cada turno. Se van comparando los valores calculados en $dp[i][j]$ con las opciones $C[i]$ y $C[j]$. Al mismo tiempo, se simula el juego para registrar las monedas elegidas por Sophia y Mateo, manteniendo el seguimiento de sus ganancias.

2.2. Ecuación de recurrencia

La ecuación de recurrencia que corresponde a este algoritmo es:

$$dp[i][j] = \max \left(\begin{array}{l} \text{monedas}[i] + \min(dp[i+2][j], dp[i+1][j-1]), \\ \text{monedas}[j] + \min(dp[i+1][j-1], dp[i][j-2]) \end{array} \right)$$

El objetivo es que Sophia maximice la suma de monedas que puede obtener, considerando que Mateo siempre elegirá la moneda más grande posible en su turno. Sea $dp[i][j]$ el valor máximo que puede obtener Sophia al jugar optimizando su elección de monedas entre las posiciones:

- Sophia elige la moneda en i , obtiene $\text{monedas}[i]$. Mateo va a elegir la mayor entre $\text{monedas}[i+1]$ y $\text{monedas}[j]$. Las opciones de Sophia quedan limitadas al intervalo $(i+2, j)$ o $(i+1, j-1)$.
- Sophia elige la moneda en j , obtiene $\text{monedas}[j]$. Mateo va a elegir la mayor entre $\text{monedas}[i]$ y $\text{monedas}[j-1]$. Las opciones de Sophia quedan limitadas al intervalo $(i+1, j-1)$ o $(i, j-2)$.

Esta ecuación de recurrencia garantiza la máxima ganancia de Sophia, ya que ella tiene en cuenta a su hermano, y aprovecha que Mateo elija la mayor moneda actual para minimizar en la medida de lo posible el resultado de él y maximizar el suyo.

2.3. Algoritmo PD

```
1 def max_ganancia(monedas):
2     n = len(monedas)
3     dp = [[0] * n for _ in range(n)]
4
5     for i in range(n):
6         dp[i][i] = monedas[i]
7
8     for longitud in range(2, n + 1):
9         for i in range(n - longitud + 1):
10             j = i + longitud - 1
11
12             if monedas[i + 1] > monedas[j]:
13                 eleccion_izq = dp[i + 2][j] if i + 2 <= j else 0
14             else:
15                 eleccion_izq = dp[i + 1][j - 1] if i + 1 <= j - 1 else 0
16
17             if monedas[i] > monedas[j - 1]:
18                 eleccion_der = dp[i + 1][j - 1] if i + 1 <= j - 1 else 0
19             else:
20                 eleccion_der = dp[i][j - 2] if i <= j - 2 else 0
21
22             dp[i][j] = max(monedas[i] + eleccion_izq, monedas[j] + eleccion_der)
23
24     reconstruir_solucion(dp, monedas)
25     return dp[0][n - 1]
26
27
28 def reconstruir_solucion(dp, monedas):
29     i, j = 0, len(monedas) - 1
30     elecciones = []
31     turno_sophia = True
32     ganancia_sophia = 0
33     ganancia_mateo = 0
34
35     while i <= j:
36         if turno_sophia: # Turno de Sophia
37             if monedas[i + 1] > monedas[j]:
38                 eleccion_izq = dp[i + 2][j] if i + 2 <= j else 0
39             else:
40                 eleccion_izq = dp[i + 1][j - 1] if i + 1 <= j - 1 else 0
```



```
42     if monedas[i] > monedas[j - 1]:
43         eleccion_der = dp[i + 1][j - 1] if i + 1 <= j - 1 else 0
44     else:
45         eleccion_der = dp[i][j - 2] if i <= j - 2 else 0
46
47     if monedas[i] + eleccion_izq == dp[i][j]:
48         elecciones.append(f"Sophia debe agarrar la primera ({monedas[i]})")
49         ganancia_sophia += monedas[i]
50         i += 1
51     else:
52         elecciones.append(f"Sophia debe agarrar la ultima ({monedas[j]})")
53         ganancia_sophia += monedas[j]
54         j -= 1
55     else: # Turno de Mateo
56         if monedas[i] > monedas[j]:
57             elecciones.append(f"Mateo agarra la primera ({monedas[i]})")
58             ganancia_mateo += monedas[i]
59             i += 1
60         else:
61             elecciones.append(f"Mateo agarra la ultima ({monedas[j]})")
62             ganancia_mateo += monedas[j]
63             j -= 1
64
65     turno_sophia = not turno_sophia
66
67     print(f"Ganancia Sophia: {ganancia_sophia}")
68     print(f"Ganancia Mateo: {ganancia_mateo}\n")
```

2.3.1. Complejidad y variabilidad de valores

La complejidad del algoritmo es $\mathcal{O}(n^2)$, siendo n el tamaño del arreglo de monedas. Aunque se realizan accesos $\mathcal{O}(1)$ la cantidad de estos es: (n^2) al probar las combinaciones de índices (i, j) .

La variabilidad en los valores de las monedas afecta a las decisiones que toma el algoritmo pero no cambia su complejidad.

Supongamos un juego con monedas uniformes (por ejemplo, todas las monedas tienen el mismo valor), el algoritmo aún evalúa todas las ramas posibles pero termina tomando decisiones arbitrarias al resolver empates frecuentes. En estos casos, el tiempo podría reducirse mínimamente. Caso contrario, con monedas altamente variables cada decisión de elección puede involucrar comparaciones más significativas, lo que podría agregar un pequeño costo constante en cada decisión.

2.4. Ejemplos de ejecución

Además de los ejemplos proporcionados por la cátedra, usamos los 3 ejemplos diseñados para la parte 1 para probar el algoritmo y, además, comparar resultados.

2.4.1. Ej. 1: [50, 200, 150, 300, 100, 250, 400]

Primer turno Sophia: debe elegir entre 50 y 400. Ya no solo considera cual es el mayor valor, sino que también tiene en cuenta las futuras opciones. Se queda con 400 ya que esto maximiza su ganancia total.

Primer turno Mateo: debe elegir entre 50 y 250. Como ahora usa Greedy, elige siempre la de mayor valor, se queda con 250

Se va repitiendo el procedimiento: 50 para Sophia, 200 para Mateo, 150 para Sophia, 300 para Mateo, 100 para Sophia.

Puntaje total Sophia: $400 + 50 + 150 + 100 = 700$

Puntaje total Mateo: $250 + 200 + 300 = 750$

2.4.2. Ej. 2: [520, 781, 334, 568, 706, 362, 201, 482, 19, 145]

Primer turno Sophia: debe elegir entre 520 y 145. Como ahora tiene en cuenta las futuras opciones, se queda con 145 ya que, si bien es la de menor valor, esto maximiza su ganancia total a futuro.

Primer turno Mateo: debe elegir entre 520 y 19. Se queda con 520.

Se va repitiendo el procedimiento: 781 para Sophia, 334 para Mateo, 568 para Sophia, 706 para Mateo, 362 para Sophia, 201 para Mateo, 482 para Sophia, 19 para Mateo.

Puntaje total Sophia: $145 + 781 + 568 + 362 + 482 = 2338$

Puntaje total Mateo: $520 + 334 + 706 + 201 + 19 = 1780$

2.4.3. Ej. 3: [45, 78, 120, 350, 95, 180, 250, 300, 400, 210, 50, 275, 330, 125, 80, 500]

Primer turno Sophia: debe elegir entre 45 y 500. Se queda con 500.

Primer turno Mateo: debe elegir entre 45 y 80. Se queda con 80.

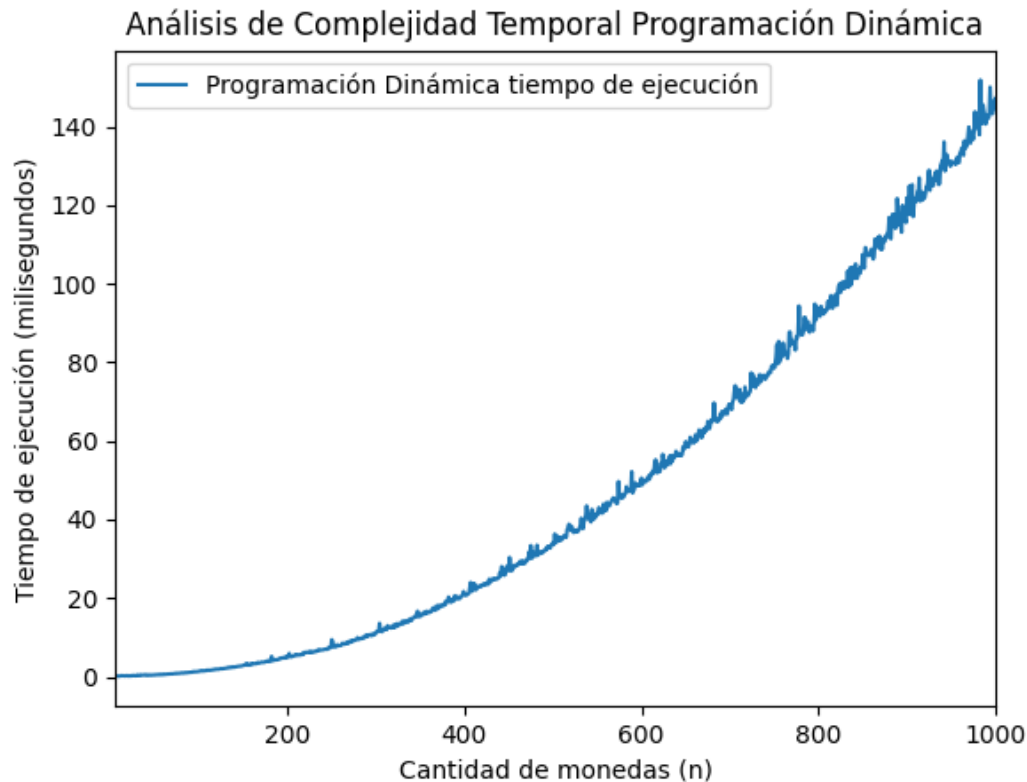
Se va repitiendo el procedimiento: 125 para Sophia, 330 para Mateo, 275 para Sophia, 50 para Mateo, 45 para Sophia, 210 para Mateo, 400 para Sophia, 300 para Mateo, 250 para Sophia, 180 para Mateo, 78 para Sophia, 120 para Mateo, 350 para Sophia, 95 para Mateo.

Puntaje total Sophia: $= 500 + 125 + 275 + 45 + 400 + 250 + 78 + 350 = 2023$

Puntaje total Mateo: $= 80 + 330 + 50 + 210 + 300 + 180 + 120 + 95 = 1365$

2.4.4. Grafico Programación dinámica

Se crearon 1000 listas de monedas de contenido aleatorio, se observa que el gráfico generado cumple con el esperado crecimiento cuadrático; a media que se aumenta la cantidad de monedas, siendo consistente con la complejidad teórica planteada ($\mathcal{O}(n^2)$).



3. Cambios: La Batalla Naval individual

3.1. El problema de la Batalla Naval se encuentra en NP

Los problemas NP (nondeterministic polynomial time) constan de una decisión que puede ser verificado en tiempo polinómico, pero no se conoce un algoritmo determinista eficiente para resolverlo. Las características en común mas relevantes de estos problemas son:

1. El número de posibilidades crece exponencialmente con el tamaño de la entrada.
2. No se conoce un algoritmo determinista eficiente que encuentre una solución óptima en tiempo polinómico.
3. La búsqueda de soluciones implica explorar un “espacio de soluciones”, donde cada solución es una combinación de los parámetros del problema.

Estas características se encuentran en el problema de la batalla naval planteado en la consigna; se tienen K barcos de B longitud que se tienen que colocar en un tablero en $N * M$ con X condiciones para cada fila e Y condiciones para las columnas, dependiendo de del tamaño de estos parámetros la dificultad crece exponencialmente, así como el tamaño de las posibilidades en el árbol de decisiones.

Para resolver el problema se uso backtracking ya que no se conoce una solución optima para resolverlo, teniendo que realizar una búsqueda exhaustiva que permite explorar el espacio de soluciones de manera sistemática retrocediendo al no cumplirse una de las condiciones que permite colocar un barco pero si se puede verificaren tiempos polinomial

Contamos con verificar solución que dado un tablero resuelto, los barcos filtrados, las restricciones de las filas y de las columnas puede verificar la solución aportada por el algoritmo de

backtracking (tablero y barcos filtrados) con una complejidad de $\mathcal{O}(n * m + k \log k)$ siendo k la cantidad de barcos esperados y $n * m$ el tamaño del tablero. $k \log(k)$ es menor a $n * m$ pero no es despreciable, en un algoritmo resuelto con las restricciones del juego es imposible que haya $n * m$ barcos pero estos ocupan una proporción no despreciable del tablero en la una partida.

```
1 def verificar_adyacencias(tablero, n, m, x, y, barco_tamano, orientacion): # 0(1)
2
3     # Direcciones de las celdas adyacentes
4     if orientacion == 'H':
5         adyacentes = [
6             (x-1, j) for j in range(y-1, y + barco_tamano + 1) if 0 <= j < m
7         ] + [
8             (x+1, j) for j in range(y-1, y + barco_tamano + 1) if 0 <= j < m
9         ] + [
10            (x, y-1), (x, y + barco_tamano)
11        ]
12     elif orientacion == 'V':
13         adyacentes = [
14             (i, y-1) for i in range(x-1, x + barco_tamano + 1) if 0 <= i < n
15         ] + [
16             (i, y+1) for i in range(x-1, x + barco_tamano + 1) if 0 <= i < n
17         ] + [
18             (x-1, y), (x + barco_tamano, y)
19         ]
20     else:
21         adyacentes = [
22             (x-1, y-1), (x-1, y), (x-1, y+1),
23             (x, y-1), (x, y+1),
24             (x+1, y-1), (x+1, y), (x+1, y+1)
25         ]
26
27     # Verificar si alguna celda adyacente contiene un barco
28     for i, j in adyacentes:
29         if 0 <= i < n and 0 <= j < m and tablero[i][j] == 1:
30             return False
31     return True
32
33 def verificar_solucion(tablero, demanda_filas, demanda_columnas, barcos_esperados):
34     n = len(tablero)
35     m = len(tablero[0])
36
37     demandas_filas_actual = [0] * n # 0(n)
38     demandas_columnas_actual = [0] * m # 0 (n)
39
40     for i in range(n): # 0(n*m)
41         for j in range(m):
42             if tablero[i][j] == 1:
43                 demandas_filas_actual[i] += 1
44                 demandas_columnas_actual[j] += 1
45
46     for i in range(n): # 0(n)
47         if demandas_filas_actual[i] > demanda_filas[i]:
48             print(f"Error: Demanda de la fila {i} excedida ({demandas_filas_actual[i]} > {demanda_filas[i]}).")
49             return False
50
51     for j in range(m): # 0(m)
52         if demandas_columnas_actual[j] > demanda_columnas[j]:
53             print(f"Error: Demanda de la columna {j} excedida ({demandas_columnas_actual[j]} > {demanda_columnas[j]}).")
54             return False
55
56     barcos_contados = []
57     visitado = [[False for _ in range(m)] for _ in range(n)]
58
59     def buscar_continuidad_barco(x, y, orientacion):
60         if x < 0 or x >= n or y < 0 or y >= m or visitado[x][y] or tablero[x][y] == 0:
61             return 0
62         visitado[x][y] = True
```

```
63     if orientacion == 'H':
64         return 1 + buscar_continuidad_barco(x, y + 1, orientacion)
65     elif orientacion == 'V':
66         return 1 + buscar_continuidad_barco(x + 1, y, orientacion)
67     return 1
68
69     for i in range(n): # O(n*m + k)
70         for j in range(m):
71             if tablero[i][j] == 1 and not visitado[i][j]:
72                 if j + 1 < m and tablero[i][j + 1] == 1: # Horizontal
73                     barco_tamano = buscar_continuidad_barco(i, j, 'H')
74                     if not verificar_adyacencias(tablero, n, m, i, j, barco_tamano,
75 'H'):
76                         print("Error: Barcos adyacentes detectados.")
77                         return False
78                     elif i + 1 < n and tablero[i + 1][j] == 1: # Vertical
79                         barco_tamano = buscar_continuidad_barco(i, j, 'V')
80                         if not verificar_adyacencias(tablero, n, m, i, j, barco_tamano,
81 'V'):
82                             print("Error: Barcos adyacentes detectados.")
83                             return False
84                         else: # Barco individual
85                             barco_tamano = 1
86                             if not verificar_adyacencias(tablero, n, m, i, j, barco_tamano,
87 '1'):
88                                 print("Error: Barcos adyacentes detectados.")
89                                 return False
90
91                 barcos_contados.append(barco_tamano)
92
93     barcos_contados.sort(reverse=True) # O(klog(k))
94     barcos_esperados.sort(reverse=True) # O(klog(k))
95
96     if barcos_contados != barcos_esperados: # O(k)
97         print(f"Error: Los barcos_esperados colocados {barcos_contados} no
98 coinciden con los barcos_esperados esperados {barcos_esperados}.")
99         return False
100
101     print("La solucion es valida.")
102     return True
```

3.2. El problema de la Batalla Naval es NP-completo

El problema planteado por la cátedra presenta características típicas de los problemas NP-completos, para ser completo debe comprobarse que esta en NP, demostrado en la sección anterior.

Para demostrar que el problema es NP-Completo, se puede reducir en tiempo polinómico un problema conocido como NP-completo al problema de la Batalla Naval.

3.2.1. Reducción desde el problema del Bin Packing

El problema del Bin Packing consiste en determinar si un conjunto de objetos de tamaños dados puede ser empaquetado en contenedores con una capacidad fija. En el caso de la Batalla Naval:

- Las restricciones de capacidad de los contenedores se traducen en las demandas de las filas y columnas.
- Las filas y columnas del tablero representan los contenedores, las restricciones de (no solapamiento, no adyacencia) son consistentes con las restricciones implícitas de Bin Packing.
- Los barcos corresponden a los objetos a empaquetar.

Reducción detallada:

- Pasos previos:

Cada objeto con un tamaño s_i en Bin Packing se convierte en un barco de longitud s_i .

Definir un tablero de mm , donde m es el número de contenedores necesarios.

Inicializar las demandas de las filas y columnas como C

- Ya preparada la instancia de bin packing como una entrada valida de BNI ya se puede utilizar el algoritmo que resuelve el juego de mesa
- Sí se logra colocar todos los barcos respetando las restricciones, la instancia de Batalla Naval es una solución válida para la instancia de Bin Packing, sino no se puede resolver.

Esta transformación a BNI se puede hacer polinomialmente

3.2.2. Código para una entrada valida de BNI

```
1 def transformar_bin_packing_a_bni(objetos, capacidad): #  $O(c^2 + o)$ 
2
3     suma_objetos = sum(objetos)
4     contenedores = (suma_objetos + capacidad - 1) // capacidad
5
6     tablero = [[0 for _ in range(contenedores)] for _ in range(contenedores)]
7     demanda_filas = [capacidad] * contenedores
8     demanda_columnas = [capacidad] * contenedores
9
10    # Barcos son los tamanios de los objetos, se copiara para mayor comprension,
11    # no es necesario para el algoritmo y no se incluire en la complejidad final
12    barcos = objetos.copy()
13
14    return tablero, demanda_filas, demanda_columnas, barcos
```

3.2.3. Reducción de BNI a Bin packing

Para probar que la reducción es correcta se debe corroborar también que BNI es reducible polinomialmente a BP (doble implicancia).

- Pasos previos: Cada barco s_i de la lista de barcos se convierte en un objeto de tamaño s_i
Para cada fila i y columna j se crea un contenedor con capacidad $\mathbb{N} \setminus \{0\}$. Esto asegura que al colocar un 'barco' en un contenedor, se reduzca la capacidad disponible tanto de la fila i como de la columna j
- Ya preparada la instancia de BNI como una entrada valida de BP ya se puede utilizar el algoritmo que resuelve la asignación de contenedores.
- Una vez que la caja negra proporciona una solución (asignación de objetos a contenedores combinados), se puede reconstruir el tablero para BNI:
Para cada barco asignado a un contenedor $C_{i,j}$ se lo coloca en la intersección de la fila i y la columna j en el tablero, se debe actualizar las capacidades restantes de r_i y c_j para asegurarte de que no se excedan. Sí alguna fila o columna excede su capacidad original, la solución no es válida.

Esta transformación a BP se puede hacer polinomialmente

3.2.4. Código para una entrada valida de BP

```
1 def instancia_BNI_a_BP(barcos, restricciones_filas, restricciones_colum): #  $O(n * m)$ 
2     contenedores = []
3
4     for i in range(len(restricciones_filas)):
5         for j in range(len(restricciones_colum)):
6             contenedores.append(min(restricciones_filas[i], restricciones_colum[j]))
7
8     # los objetos son equivalentes a los barcos. Se creo objetos
9     # para mejor comprension de la transformacion y no es necesario
10    # ni lo incluiremos en la complejidad
11    objetos = barcos.copy()
12
13    return objetos, contenedores
```

Como conclusión es posible transformar cualquier instancia del problema de Bin Packing en una configuración del problema de la Batalla Naval en tiempo polinómico. Por lo tanto, reduciendo desde problemas conocidos como el bin packing.

$$BPacking \leq_p BNIindividual$$

3.2.5. ¿Bin packing es NP-Completo?

Verificar si esta asignación es válida (es decir, que cada contenedor no exceda su capacidad y que el número de contenedores sea k o menos) se puede hacer en tiempo polinomial; Bin packing esta en NP. Demostraremos que Bin packing es efectivamnete NP-C con otra reducción que, por la propiedad que tienen las reducciones de ser transitivas, podremos concluir que:

$$PARTITION \leq_p BPacking \leq_p BNIindividual$$

PARTITION: Dado un conjunto S ¿Se puede dividir S en 2 conjuntos tal que la sumatoria de los elementos de S_1 sea igual a sumatoria de los elementos de S_2 ? (Dicho de otra forma elementos tanto de S_1 y S_2 deben sumar suman exactamente $\sqcup$, donde $\sqcup$ es la suma de todos los elementos de S)

PARTITION es un conocido problema de decisión NP-C y se puede reducir a Bin packing:

Sea el subconjunto $S \{x_1, x_2, \dots, x_n\}$ al cual como paso previo para preparar entrada a bin tracking debemos normalizar los elementos dividiendo cada x_i por T . Establecemos $k = 2$ y la capacidad de cada contenedor como 1.

Ya con el resultado de BP , sí podemos empaquetar todos los elementos en $k = 2$ contenedores, entonces existe una partición de S tal que la suma de los elementos en cada subconjunto es igual a $T/2$.

Si no es posible empaquetar los elementos en 2 contenedores, entonces no existe tal partición. La reducción de $PARTITION$ a BP se realiza en tiempo polinomial (normalización de los elementos y configuración de la instancia de BP) implicando que:

$$PARTITION \leq_p BPacking \leq_p BNIindividual$$

3.2.6. Conclusión

El problema de la Batalla Naval es NP-completo porque:

1. Pertenece a NP, ya que la validez de una solución propuesta puede verificarse en tiempo polinómico.
2. Es NP-Complejo, ya que problemas NP-completos, como el Bin Packing o Partition (o el 3-Partition mencionado en el enunciado, siendo este ultimo una variación de Partition) , pueden reducirse en tiempo polinómico al problema de la Batalla Naval.

Por lo tanto, no existe un algoritmo eficiente conocido para resolver el problema en todos los casos.

3.3. Back-tracking

Los algoritmos de backtracking prueban todas las opciones de un árbol de decisión que, inevitablemente, tendrá repeticiones. Para suplir eso se deben 'podar' ramas innecesarias o repetidas y amortizar la complejidad que tendría un 'peor caso', por ejemplo un $n * m$ tablero y con barcos imposibles de colocar.

```
1 def solucion_backtracking(demanda_filas, demanda_columnas, barcos):
2     n = len(demanda_filas)
3     m = len(demanda_columnas)
4
5     tablero = [[0 for _ in range(m)] for _ in range(n)]
6
7     barcos.sort(reverse=True)
8     barcos_procesados = filtrar_barcos_imposibles(demanda_filas, demanda_columnas,
9     barcos)
10
11     tablero_obtenido = batallanaval(tablero, demanda_filas, demanda_columnas,
12     barcos_procesados)
```

3.3.1. Explicación de las funciones usadas en el algoritmo

```
1 filtrar_barcos_imposibles(demanda_filas, demanda_columnas, barcos):
```

El propósito de esta función es filtrar los barcos que no pueden colocarse en el tablero debido a restricciones de demanda. Para cada barco, verifica si su longitud puede satisfacer al menos una fila y una columna de las demandas actuales. Si esto es posible, el barco se incluye en la lista de barcos procesados.

Resultado: Devuelve una lista de barcos que son posibles de colocar dadas las demandas iniciales.

```
1 batallanaval(tablero, demanda_filas, demanda_columnas, barcos):
```

Luego del primer filtro resuelve el problema del tablero utilizando backtracking, colocando los barcos en el tablero de manera válida y maximizando las demandas satisfechas: Inicializa un tablero vacío con las dimensiones dadas por las demandas de filas y columnas. Ordena los barcos de mayor a menor longitud para intentar colocar los más grandes primero. Llama al proceso de backtracking para intentar ubicar los barcos en todas las posiciones y orientaciones posibles.

Resultado: Devuelve el mejor tablero encontrado donde los barcos colocados maximizan la cantidad de demandas satisfechas.

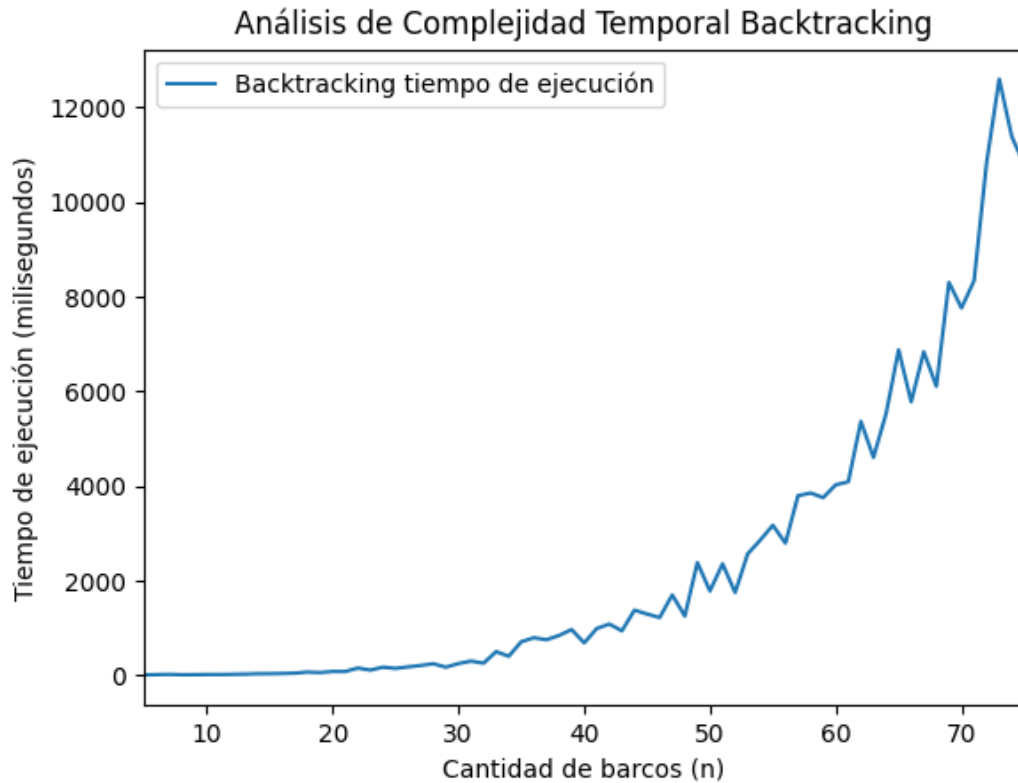
3.3.2. Complejidad algorítmica

En el algoritmo planteado para resolver el juego de Sophia tiene una complejidad $\mathcal{O}((n * m)^k)$ dado a que se realizan podas a muchas combinaciones imposibles o repetidas se eliminan, reduciendo significativamente el espacio de búsqueda. La complejidad depende mucho de n , m (tablero) y k (cantidad de barcos) por lo que el algoritmo es ineficiente para valores grandes de estas debido a su naturaleza combinatoria. Aún así se tiene una buena optimización ya que sin podas la complejidad sería: $\mathcal{O}((2 * n * m)^k)$, sin estas el crecimiento exponencial es menos manejable, y el algoritmo rápidamente se vuelve ineficiente incluso para tableros medianos o un número moderado de barcos.

3.3.3. Grafico Backtracking

Se crean 75 juegos aleatorios (barcos, condiciones de filas y columnas) y se repite 3 veces para un promedio con más precisión de los tiempos medidos, evitando el efecto de posibles fluctuaciones

en los tiempos de ejecución debido a factores externos, como carga del sistema. El gráfico generado presenta un crecimiento exponencial esperado para un algoritmo de backtracking.



3.4. Aproximación

Se propuso un algoritmo de aproximación el cual itera los barcos de mayor a menor largo, buscando ubicar dicho barco en la fila/columna con mayor demanda en una posición valida, en caso de no poder se continua con el siguiente barco.

Utilizando la misma inicializan que en la solución de backtracking, se reemplaza el llamado dentro de batalla_naval por el que llama a un algoritmo de tipo greedy con el siguiente código:

```
1 def batalla_naval_aproximada(tablero, demanda_filas, demanda_columnas,
2   barcos_procesados): # 0 (k * ( ( (n-1)*1 + (m-1)*1 ) + n + m ) )
3   for barco in barcos_procesados: # 0 (k * ( ( (n-1)*1 + (m-1)*1 ) + n + m ) )
4       fila_max_d = demanda_filas.index(max(demanda_filas)) # 0(n)
5       col_max_d = demanda_columnas.index(max(demanda_columnas)) # 0(m)
6
7       if(demanda_filas[fila_max_d] == 0 and demanda_columnas[col_max_d] == 0):
8           return tablero
9       if(barco <= demanda_filas[fila_max_d] or barco <= demanda_columnas[
10  col_max_d]):
11      tablero, demanda_filas, demanda_columnas, barco_colocado = poner_barco(
12  tablero, demanda_filas, demanda_columnas, barco, fila_max_d, col_max_d)
13  return tablero, demanda_filas, demanda_columnas
```

3.4.1. Explicación de las funciones usadas en el algoritmo

```
1  poner_barco(tablero, demanda_filas, demanda_columnas, barco, fila_max_d,
2  col_max_d):
```

Esta es la función principal del algoritmo, dentro de ella hace una serie de validaciones. En caso de que la demanda mas alta este en una fila, prueba colocar el barco en ella, si fuera el caso de que la demanda mas alta este en una columna, prueba colocarlo en esta ultima. En caso de que la demanda máxima sea igual para una fila y una columna, se prueba primero por fila y si esto no es posible, se prueba sobre la columna.

Para validar si se puede colocar o no un barco en una fila/columna, se verifica si el largo es valido para las demandas que involucra. En caso de que sea valida la colocación, se pone el barco y actualiza las demandas

3.4.2. Complejidad algorítmica

La complejidad algorítmica del algoritmo de aproximación propuesto es de $\mathcal{O}(k * ((n - l) * l + (m - l) * l))$ teniendo al termino de la complejidad $\mathcal{O}((n - l) * l)$ dado al colocar un barco en forma vertical y al termino $\mathcal{O}((m - l) * l)$ para colocar al barco de forma horizontal. Dichas complejidades vienen dado de que se itera el barco de largo l sobre una fila/columna. La iteración es desde el inicio de dicha fila/columna hasta que el largo del barco este sobre el limite de la matriz, de ahí el termino $(m - l)$ o $(n - l)$. Dentro de esta iteración, se valida el barco que tiene una complejidad de $\mathcal{O}(l)$, en caso de que sea valido se lo coloca con un costo de $\mathcal{O}(l)$ y actualizan las demandas con el mismo orden temporal, dando esto el orden previamente mencionado.

3.4.3. Análisis de la Cota de Aproximación

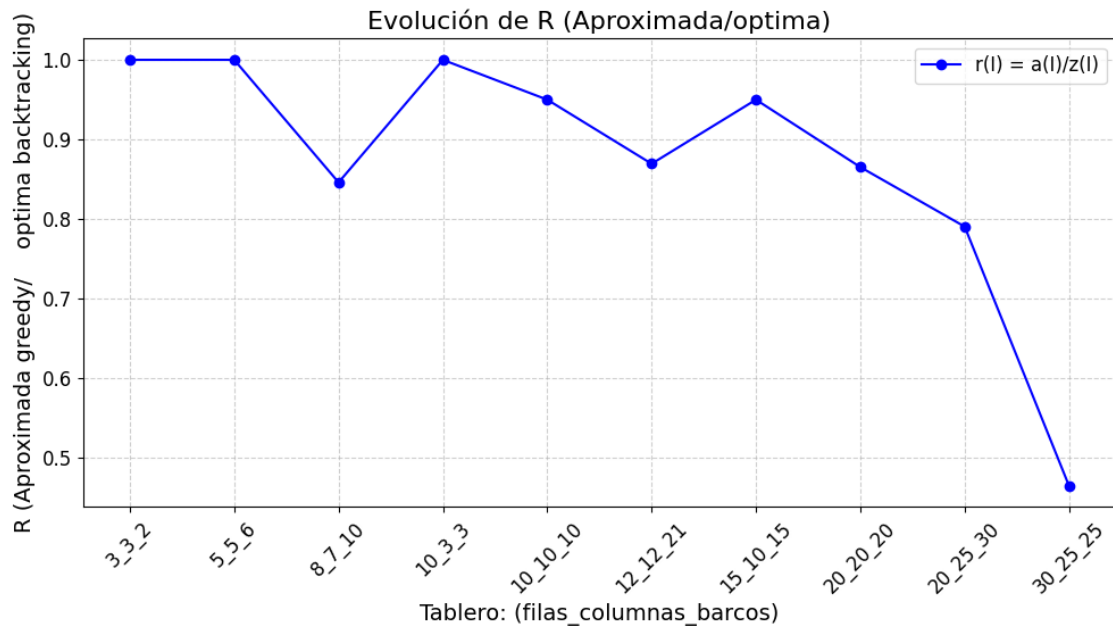
Siendo I una instancia del problema, $z(I)$ una solución optima de dicha instancia, en este caso calculada por backtracking y $A(I)$ una solución aproximada, se define $A(I)/z(i) \leq r(A)$. En la tabla siguiente se tiene las diferentes instancias de ejemplos dada por la cátedra, teniendo el calculo de $r(A)$. En dicha tabla se puede observar que va decreciendo $r(A)$ al aumentar el tamaño de las variables n, m y k .

I	n	m	k	$z(I)$	$A(I)$	$r(A)$
1	3	3	2	4	4	1
2	5	5	6	12	12	1
3	8	7	10	26	22	0.84
4	10	3	3	6	6	1
5	10	10	10	40	38	0.95
6	12	12	21	46	40	0.86
7	15	10	15	40	38	0.95
8	20	20	20	104	90	0.86
9	20	25	30	172	136	0.79
10	30	25	25	202	94	0.46
Promedio	—	—	—	—	—	0.871

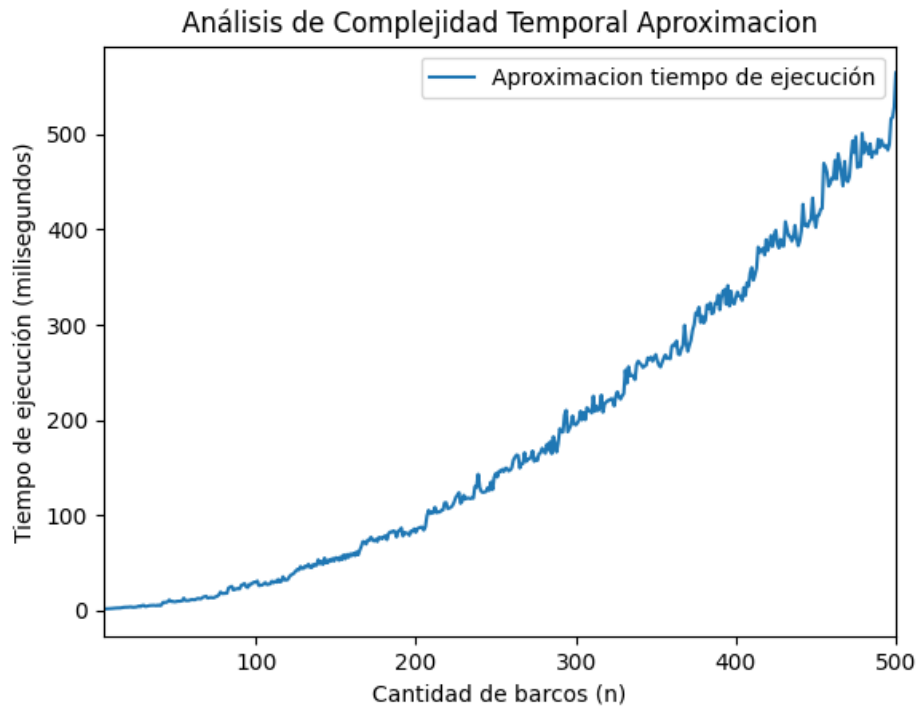
Cuadro 1: Tabla con valores de los ejemplos de la cátedra.

3.4.4. Graficos Aproximación

En la siguiente figura se comparan los diferentes valores de $r(A)$ que se obtiene comparando la solucion optima obtenida por backtracking contra la solución aproximada que se obtiene del algoritmo greedy propuesto.



Siguiendo las mismas mediciones que en el algoritmo optimo, se crean 500 juegos aleatorios (barcos, condiciones de filas y columnas), repitiéndolo 3 veces para obtener un promedio de los tiempos medidos para evitar fluctuaciones por factores externos. El gráfico generado presenta un crecimiento lineal esperado por la cota superior obtenida en el análisis de la complejidad temporal del algoritmo propuesto ($\mathcal{O}(k * ((n - l) * l + (m - l) * l) + n + m)$). Además, se puede observar que los tiempos obtenidos son de varios ordenes de magnitud menos que en la misma medición para el algoritmo optimo.



4. Conclusiones

4.1. Greedy

La facilidad de los algoritmos recuerda a el algoritmo de ordenamiento de selección; es fácil de comprender y es el que se usa en la vida diaria pero, por este motivo, la no da la talla para la parte 2 o la parte 3 dado a que se requiere un pensamiento más profundo en cuanto a las posibles acciones a tomar porque los greedy solo concederán la mejor opción actual.

4.2. Programación dinámica

El algoritmo implementado permite a Sophia maximizar su puntaje total al prever las elecciones de Mateo, quien sigue una estrategia greedy. Esto se logra utilizando una matriz dp para almacenar los resultados óptimos de subproblemas, aprovechando la ecuación de recurrencia diseñada para modelar las interacciones estratégicas entre ambos jugadores, diferenciándose del algoritmo de la parte 1 que tomaba decisiones mas simples.

La complejidad $\mathcal{O}(n^2)$ del algoritmo resulta manejable para tamaños moderados del conjunto de monedas, y su implementación garantiza un equilibrio entre eficiencia computacional y precisión en los resultados.

La necesidad de seguir ganando de Sophia después de que Mateo aprenda a jugar muestra la importancia de modelar problemas complejos con herramientas avanzadas como la programación dinámica, pudiendo ser aplicadas a escenarios reales donde se requiere optimización en contextos competitivos.

4.3. Back-Tracking

El problema de la batalla naval es un ejemplo práctico de cómo los problemas NP se encuentran en escenarios cotidianos, como juegos o configuraciones espaciales y múltiples restricciones simultáneamente generan un espacio de soluciones altamente complejo. El backtracking ofrece soluciones exactas a un alto costo algorítmico y difíciles de seguir, siendo imposible para el programador realizar el seguimiento de todos los casos posibles. Como sugiere el enunciado, alternativas como métodos heurísticos, algoritmos genéticos o aproximaciones con programación lineal entera podría resolver de mejor forma el problema de la batalla naval.

4.4. Aproximación

Los algoritmos de aproximación pueden ser una mejor solución que una solución optima en algunos contextos, dado que tienen una mejor complejidad temporal frente a la complejidad exponencial en problemas combinatorios como el de la Batalla Naval Individual. En problemas de gran escala, una solución optima puede llegar a ser impracticable ya sea por tiempo o recursos computacionales.

Aunque los algoritmos de aproximación no garantizan una solución optima, pueden proporcionar resultados cercanos a los óptimos, en este caso un valor de $r(A) \geq A(I)/z(i)$